Aran Learn 11: Frontend Basics

For: everyone. Inbathamizhan leads the console; Bharadhwaj and Arun own the detector and immune pages; Aasif owns the erasure and leak-trace pages. **Time:** about 5 hours with practice. **Before this:** Learn 8 (HTTP, JSON, FastAPI). You need Node.js 20 or later (`node -v`) and Python 3.12 or later.

The Aran console in `app/` is a real website: a **frontend** (what runs in the browser) talking to a **backend** (the FastAPI server running Aran's firewall). Learn 8 covered the backend side. This doc covers the frontend: how a browser draws a page, how React builds the page out of components, and how the page asks the server for live data. By the end you can add a new panel to the console yourself.

1. What happens when you open a website

1. The browser asks the server for `index.html` (an HTTP `GET`, Learn 8 section 1).
2. The HTML names a stylesheet (CSS) and a script (JavaScript). The browser downloads both.
3. The browser builds the **DOM** (Document Object Model): a tree of boxes, one per HTML element.
4. CSS decides how each box looks: colour, size, position.
5. JavaScript runs. It can change the DOM at any time, and it can make more HTTP requests to fetch data. This is how the console updates its numbers without reloading the page.

So there are three languages, each with one job:

Language	Job	Aran example
HTML	Structure: what is on the page	a table of findings
CSS	Looks: colours, spacing, layout	green underline on real values, amber on tokens
JavaScript	Behaviour: what happens when you click, fetching data	"Send sample request" calls the API and shows the result

2. HTML in five minutes

HTML is a tree of **elements**. Each has an opening tag, content and a closing tag:

```
<section class="card">
  <h3>Findings</h3>
  <table>
    <tr><td>Aadhaar</td><td>98.7%</td></tr>
```

```
      </table>
      <button>Trace the leak</button>
    </section>
```

- `class="card"` is an **attribute**. CSS uses the class to style every element marked `card` the same way.
- Use the element that matches the meaning: `<button>` for actions, `<a>` for links, `<h1>`–`<h3>` for headings, `<table>` for tables. Screen readers and keyboards rely on this; it is also half of accessibility.

3. CSS in ten minutes

A CSS **rule** is a selector plus declarations:

```
    .card { background: var(--surface); border: 1px solid var(--border); border-radius: 10px; padding: 20px; }
    mark.m-token { background: var(--token-bg); }
```

- `.card` selects every element with class `card`.
- `var(--surface)` reads a **custom property**, our design token. The console defines every colour once at the top of `app/web/src/styles.css`, twice: once for light theme and once for dark. Changing the theme only swaps the token values; no rule changes. This is the same idea as the design system in the Claude Design guide.
- **Layout.** Two tools do almost everything: `display: flex` lays children in a row or column; `display: grid` lays them on a grid. `.grid4 { display: grid; grid-template-columns: repeat(4, 1fr) }` gives the four stat tiles on the Overview page.
- **Responsive.** `@media (max-width: 760px) { ... }` applies rules only on narrow screens. That is how the sidebar becomes a slide-out menu on a phone.

Rule of thumb for Aran: green, amber and red are only for real, token and leak. Everything else uses the neutral tokens.

4. JavaScript in fifteen minutes

JavaScript looks like a mix of Python and C:

```
    const findings = [{ type: 'AADHAAR', p: 0.987 }, { type: 'OTP', p: 0.42 }]
    const caught = findings.filter(f => f.p >= 0.5)          // like a Python list comprehension
    console.log(caught.map(f => f.type))                      // [ 'AADHAAR' ]
```

Python	JavaScript
`x = 5`	`const x = 5` (cannot be reassigned) or `let x = 5`

Python	JavaScript
`def f(a): return a * 2`	`const f = a => a * 2`
`[f(x) for x in xs if x > 0]`	`xs.filter(x => x > 0).map(f)`
`{"a": 1}["a"]`	`({ a: 1 }).a`
`f"{n} found"`	`` `${n} found` ``
`None`	`null` or `undefined`

Try it: open the browser's developer tools (F12), go to the Console tab and paste the snippet.

Asking the server for data: `fetch`

A web page talks to an API with `fetch`. It returns a **promise**: a value that arrives later. `await` waits for it without freezing the page.

```
const res = await fetch('/api/stats')
const stats = await res.json()
console.log(stats.requests, stats.p95_ms)
```

Sending data is a `POST` with a JSON body, exactly the request Learn 8 built with FastAPI:

```
const res = await fetch('/api/process', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ text: 'uid=234123412346', source: 'my-app', sink: 'log-
platform' }),
})
const ev = await res.json()
console.log(ev.after, ev.findings)
```

The console wraps this in one helper, `api(path, body)` in `app/web/src/api.js`, so every page calls `api('/stats')` or `api('/trace', { dump })`.

5. React: building pages from components

Writing `document.createElement` by hand for every table row gets painful fast. **React** lets you describe what the page should look like for the current data, and it updates the DOM for you when the data changes.

Components and JSX

A component is a function that returns markup. The markup is **JSX**: HTML-like syntax inside JavaScript. Curly braces `{}` drop back into JavaScript.

```jsx
function Stat({ label, value, note }) {
  return (
    <div className="stat card">
      <div className="stat-v mono">{value}</div>
      <div className="stat-l">{label}</div>
      {note && <div className="stat-n">{note}</div>}
    </div>
  )
}

// used like an HTML element:
<Stat label="Requests protected" value={stats.requests} note="live" />
```

- `label`, `value`, `note` are **props**: inputs passed in by the parent, like function arguments.
- In JSX it is `className`, not `class`.
- `{note && <div>...}` means "only show this if `note` is set".
- To repeat something, map a list to elements, each with a unique `key`:

```jsx
{findings.map((f, i) => <tr key={i}><td>{f.type}</td><td>{f.p}</td></tr>)}
```

State: data that changes

`useState` gives a component memory. When you call the setter, React redraws that component.

```jsx
import { useState } from 'react'

function TraceBox() {
  const [dump, setDump] = useState('')
  const [result, setResult] = useState(null)
  const trace = async () => setResult(await api('/trace', { dump }))
  return (
    <>
      <textarea value={dump} onChange={e => setDump(e.target.value)} />
      <button onClick={trace} disabled={!dump}>Trace the leak</button>
      {result && <p>Most likely source: {result.top?.sink}</p>}
    </>
  )
}
```

That is the whole pattern of every console page: inputs update state, a button calls the API, the answer goes into state, React redraws.

Effects: doing something when the page opens

`useEffect` runs code after React draws, for example to load data or start a timer. The console's `usePoll` hook (in `api.js`) uses it to refresh numbers every few seconds:

```
  useEffect(() => {
    load()                                  // once now
    const t = setInterval(load, 3000)       // then every 3 s
    return () => clearInterval(t)           // stop when the page closes
  }, [load])
```

Then a page simply writes `const [stats] = usePoll('/stats', 3000)`.

6. The tools: npm and Vite

- **npm** installs JavaScript libraries listed in `package.json` into `node_modules/`, like `pip install -r requirements.txt`.
- **Vite** is the development server and build tool. In development it serves the page and reloads instantly on save. For production, `npm run build` turns the JSX into plain JavaScript and CSS files in `dist/`.
- **The proxy.** In development the page runs on port 5173 and the API on port 8000. `vite.config.js` forwards every `/api` request to port 8000, so the code can always call `/api/...`. In production the FastAPI server serves both the API and the built `dist/` files, so there is only one address.

7. Run the Aran console on your laptop

```
  # once: data and the trained models
  cd research && ./fetch_data.sh && ./fetch_ai_data.sh && cd ..
  python3 -m venv .venv && source .venv/bin/activate
  pip install -r app/server/requirements.txt
  python3 app/server/build_models.py          # only if
app/server/models/detector.pkl is missing

  # terminal 1: the API (real firewall)
  cd app/server && uvicorn main:app --reload --port 8000

  # terminal 2: the frontend
  cd app/web && npm install && npm run dev     # open http://localhost:5173
```

Open http://localhost:8000/docs as well: FastAPI lists every endpoint the console uses and lets you call them by hand. This is the fastest way to see what data a page gets.

8. How the console is organised

```
app/
  server/
    main.py        FastAPI routes: /api/stats, /api/sample, /api/trace, /api/erase, ...
    engine.py      the live engine: detector + firewall + vault + immune layer + replay
```

```
  build_models.py  trains the detector once and saves models/detector.pkl
web/
  src/api.js         api() and usePoll()
  src/components.jsx Stat, IdStrip, Marked (before text), Tokens (after text), Bar
  src/pages/*.jsx    one file per page in the sidebar
  src/styles.css     design tokens and all styles
Dockerfile       builds the frontend, installs the server, and runs both as one service
```

Page	File	API calls
Overview	`Overview.jsx`	`/stats`, `/events`, `/immune`
Live traffic	`Traffic.jsx`	`/config`, `/sample`, `/process`, `/events`
Customers & erasure	`Customers.jsx`	`/customers`, `/erase`, `/receipts/verify`, `/detokenise`, `/merkle`
Leak trace	`Trace.jsx`	`/leaked-dump`, `/trace`
Immune layer	`Immune.jsx`	`/immune`, `/immune/learn`, `/replay/*`, `/drills/*`, `/immune/alerts/*/label`
Results & tests	`Results.jsx`	`/results`, `/jobs/tests`, `/jobs/e1-e4`

9. From Claude Design to this code

When the team finishes a design in Claude Design and exports it, do not throw away this code. The design changes how things look; the API calls stay the same. In practice:

1. Copy the new colours, fonts and spacing into the tokens at the top of `styles.css`.
2. Rebuild each page's markup to match the design, keeping each page's `api(...)` and `usePoll(...)` calls.
3. Check light theme, dark theme and phone width (about 390 px) before you push.

10. Accessibility checklist

- Every action is a `<button>`, every navigation a link, so the keyboard reaches it with Tab.
- Visible focus ring on everything (`:focus-visible` in `styles.css`).
- Colour is never the only signal: a token cell also has a slash mark and a label, a raw value says "passed raw".
- Text contrast at least 4.5:1 in both themes.
- Respect `prefers-reduced-motion`.

Practice

1. **Read the data.** With the server running, open http://localhost:8000/api/stats in the browser. Find `p95_ms` and `values_protected`. Send one sample request on the Live traffic page and reload the JSON. Which numbers changed?
2. **Add a stat tile.** On the Overview page, add a fifth tile “Vault tokens” that shows `s.vault_tokens`. (Hint: copy one `<Stat ... />` line.)
3. **Add a column.** In the Findings table on Live traffic, add a column “Rule” that shows `f.evidence.checksum === 1 ? 'checksum' : '—'`.
4. **New panel.** On the Leak trace page, after a trace, show the number of values that matched no sink. (Hint: `res.items.filter(x => !x.sink).length`.)
5. **Responsive check.** In developer tools, switch to a 390 px wide phone view. Find one thing that does not fit and fix it with a rule inside the `@media (max-width: 760px)` block.

Solutions

1. `requests` goes up by 1; `values_protected` goes up by the number of findings at or above the sink’s threshold; `p95_ms` may change because the new request’s latency joins the list.
2. `<Stat label="Vault tokens" value={fmt(s.vault_tokens)} note="purpose-bound, erasable" />` inside a `.grid4` block (a grid of four wraps a fifth tile onto a new row).
3. Add `<th>Rule</th>` to the header row and `<td>{f.evidence.checksum === 1 ? 'checksum' : '—'}</td>` to each body row.
4. `<p className="small muted">{res.items.filter(x => !x.sink).length} values matched no sink</p>` inside the result card.
5. Any overflow fix is fine, as long as the page never scrolls sideways; wide tables already sit in a `.scroll` box.